

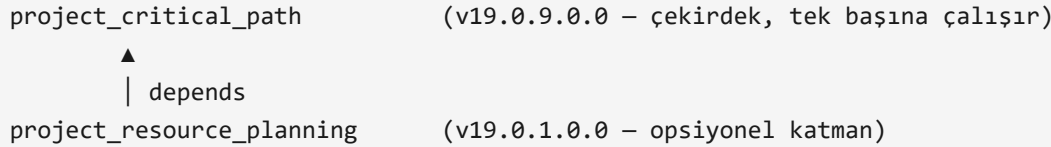
Teknik Dokümantasyon — project_critical_path +

project_resource_planning

Odoo 19 addon çifti. Bu doküman, modüllerin mimarisini, veri modelini, algoritmalarını, API yüzeyini ve frontend bileşenlerini AI analizi için kod seviyesinde özetler. Alan/metot adları koddaki haliyle bırakılmıştır.

1. Genel Mimari

İki addon tek yönlü bağımlılıkla çalışır:



- **Çekirdek** `project`, `hr_timesheet`, `web_gantt` 'e bağlıdır. CPM (Critical Path Method), WBS, baseline, delay impact ve Planner Workspace'in tamamını içerir.
- **Kaynak katmanı** çekirdeğe ek olarak `hr`, `maintenance`, `product`, `stock` 'a bağlıdır. Rol/requirement/assignment, Resource Board, Material Plan ve stok entegrasyonunu ekler.
- Çekirdek, kaynak addon'unun **yokluğunda da aynı payload şemasını** üretir: `_get_planner_resource_data`, `_planner_resource_fields`, `_get_baseline_resource_vals`, `_get_task_resource_snapshot` gibi hook metotlar çekirdekte boş/sıfır döner; kaynak addon bunları override eder. UI tarafında `get_planner_projects().has_resource_planning` bayrağı (registry'de `project.task.resource.requirement` modelinin varlığı) resource arayüzünü açıp kapatır.
- Her iki addon da `application: False`, `license: LGPL-3`.

BRD referansları: kod yorumlarındaki "BRD/BRD-XX" etiketleri iş gereksinim dokümanındaki madde numaralarına işaret eder.

2. project_critical_path — Çekirdek Modül

2.1 Veri Modeli

Yeni modeller

Model	Amaç
<code>project.critical.path</code>	Hesaplanmış bir maksimum-sürelili görev zinciri (sonuç satırı)

Model	Amaç
<code>project.critical.path.baseline</code>	Değiştirilemez plan anlık görüntüsü (revizyon)
<code>project.critical.path.baseline.line</code>	Baseline'a ait görev seviyesi donmuş değerler
<code>project.critical.path.change</code>	Görevin kritik yola giriş/çıkış kaydı
<code>project.task.delay.impact</code>	Güncel gecikme etki analizi satırları (en yeni baseline'a göre)
<code>project.task.dependency</code>	Bağımlılık kenarının nitelikleri (FS/SS/FF/SF + lag)
<code>project.baseline.title</code>	Baseline kaydında seçilebilen ön-tanımlı başlıklar (≤ 15 karakter, <code>active</code>)

`project.task` uzantısı (CPM alanları — hepsi `readonly`)

Alan	Tip	Açıklama
<code>critical_early_start/finish</code>	Float	CPM ileri geçiş sonucu (saat birimi)
<code>critical_late_start/finish</code>	Float	CPM geri geçiş sonucu
<code>critical_slack</code>	Float	<code>late_start - early_start</code> (toplam bolluk)
<code>is_critical</code>	Boolean	<code>abs(slack) < 1e-6</code>
<code>delay_baseline_duration</code>	Float	Baseline'daki süre
<code>delay_duration_variance</code>	Float	Güncel – baseline süre
<code>delay_project_impact</code>	Float	Proje bitişine etkisi (slack düşülerek)
<code>delay_impact_status</code>	Selection	<code>critical_impact / within_slack / duration_reduced / no_impact</code>
<code>delay_impact_chain</code>	Text	Okunabilir downstream zincir metni
<code>date_done</code>	Datetime (compute, store)	<code>state == '1_done'</code> iken damgalanır, reopen'da temizlenir
<code>finish_variance_state</code>	Selection (compute, store)	<code>late / early / on_time</code> — takvim günü kıyaslaması, saat bileşeni sayılmaz

`project.task` uzantısı (WBS alanları — `compute+store`, `copy=False`)

Alan	Açıklama
<code>wbs_code</code>	<code>1</code> , <code>1.1</code> , <code>1.2.3</code> ... hiyerarşik kod
<code>wbs_level</code>	Derinlik (kök=1)
<code>wbs_sort_key</code>	<code>0001.0003.0002</code> formatında sıfır-dolgu sıralama anahtarı; <code>_order = "wbs_sort_key, sequence, id"</code>
<code>is_work_package</code>	<code>bool(child_ids)</code>
<code>planned_hours_rollup</code>	Alt ağacın <code>allocated_hours</code> toplamı
<code>effective_hours_rollup</code>	Alt ağacın <code>effective_hours</code> toplamı

Alan	Açıklama
progress_rollup	Saat-ağırlıklı ($\sum \text{progress} \cdot \text{hours} / \sum \text{hours}$); saat 0 ise düz ortalama

project.project uzantısı

critical_path_count, critical_path_duration, critical_path_ids, critical_path_baseline_ids/count, delay_impact_baseline_id, delay_baseline_duration, delay_current_duration, delay_total, delay_impact_line_ids, ve planning_precision (hour|day — görüntü/girdi hassasiyeti; saklama ve motor hep datetime).

project.task.dependency (kenar nitelikleri)

- task_id, depends_on_id, project_id (related, store)
- relationship_type: fs|ss|ff|sf (varsayılan fs)
- lag + lag_unit (hours|days) → lag_hours (compute, store; gün × takvim hours_per_day). Pozitif lag = bekleme, negatif = lead.
- UNIQUE(task_id, depends_on_id) constraint'i.
- Yapısal gerçek kaynak depend_on_ids M2M'sidir.** Bu model sadece kenar niteliği taşır; _ensure_dependency_records() her hesaplamada M2M ile reconcile eder (eksikleri sudo ile yaratır, artıkları siler).

project.critical.path.baseline (immutability)

- write() / unlink() korumalı alanlarda UserError fırlatır — snapshot gerçekten değişmezdir.
- Alanlar: revision_number, name (v1.N [- etiket]), project_duration, critical_path_duration, critical_path_signature (sıralı task_path listesi), critical_path_snapshot (JSON: task_ids + isimler), previous_baseline_id, history_* alanları (önceki baseline'la karşılaştırma), planned_resource_hours/cost + currency_id.
- Compute alanları current_* / critical_path_changed güncel planla canlı karşılaştırma yapar.

2.2 CPM Algoritması (project_project.py)

_recalculate_critical_paths() :

- _get_task_dependency_graph() — projenin tüm görevleri (active_test=False). Her görev kendi allocated_hours 'u ile **schedulable düğüm**; WBS hiyerarşisi grafiği etkilemez (sadece sunum). Self-link'ler düşülür. Kenarlar tip+lag taşır. Kahn topolojik sıralama; kalan düğüm varsa **cycle** → **UserError**.
- _calculate_task_schedule() — ileri geçiş:
- EF(task) = max(duration, max over preds of edge_early_bound)
- Kenar bound'ları: FS → pred_ef + lag + dur ; SS → pred_es + lag + dur ; FF → pred_ef + lag ; SF → pred_es + lag
- project_duration = max(EF) yaprak düğümlerde
- Geri geçiş aynı kenar tipleriyle LF üst sınırlarını verir
- Görev değerleri: ES/EF/LS/LF/slack/is_critical — **değişmeyen satır UPDATE edilmez** (write hacmini düşürür, 1e-6 tolerans)

8. Kritik yol çıkarımı: `duration_by_task = early_finish` ; her düğüm için bound'u EF'e eşit olan predecessor'lar `best_predecessors` — **tüm beraberlikler korunur**. Kritik uçlardan iterative DFS ile (recursion limit'i aşmak için stack) tüm yollar bulunur, `project.critical.path` satırları eskileri silinip yeniden yazılır (CP-01 , CP-02 ...).
9. Ardından `_recalculate_delay_impacts()` , `_recalculate_critical_path_changes()` (tüm baseline'lar), `_recalculate_history_comparisons()` çağrılır.

Tetik noktaları: `project.task.create/write/unlink` içinde `{project_id, parent_id, allocated_hours, planned_date_begin, depend_on_ids, dependent_ids, date_assign, date_deadline}` kesişimi. `cp_skip_recalc` ve `cp_skip_auto_schedule` context flag'leri iç yazışları/re-entrance'ı keser.

2.3 Otomatik Çizelgeleme (`_schedule_dependents`)

BRD Dependency-Based Auto Scheduling. `date_assign` / `date_deadline` değişince:

1. Değişen görev + tüm downstream'i BFS ile `check` setine al.
2. Topolojik sırada, **datetime hassasiyetinde** bound kontrolü: FS→ `succ.start ≥ pred.stop+lag` , SS→ `≥ pred.start+lag` , FF→ `succ.stop ≥ pred.stop+lag` , SF→ `≥ pred.start+lag` .
3. İhlal eden görev, span'ini koruyarak en sıkı bound'a ileri kayar (geri kaydırma yok). Shift'ler `cp_skip_auto_schedule` ile yazılır — tek geçişte cascade.
4. **WBS parent'lar muaf**: pencereleri `_sync_parent_window` ile çocuklardan roll-up edilir; kendi dependency bound'ları için kaydırılmaz ama successor'lar parent'ın güncel penceresine bakar.
5. Sonunda tek `_recalculate_critical_paths()` .

`_sync_parent_window()` : `parent.start = min(children.date_assign)` , `parent.stop = max(children.date_deadline)` — köke kadar kabarcıklanır, skip flag'lerle.

2.4 `allocated_hours` Senkronizasyonu

Odoo 19'da `allocated_hours` planner dışı yazışlarda 0/stale kalabilir.

`_resync_allocated_hours()` : tarih penceresi olan, `allocated_hours == 0` olan **yaprak** görevlere pencereden süre türetir — aynı gün → gerçek saat farkı; çok günlü → $(\text{gün farkı}+1) \times \text{takvim hours_per_day}$ (varsayılan 8). Sıfır-olmayan değer authoritative'dir, asla ezilmez (inspector'dan gelen kesirli süreler korunur). Migration `19.0.9.0.0` bu backfill'i eski kayıtlar için bir kez çalıştırır.

`update_planner_task()` tarafında: `dt_start/dt_stop` çifti geldiğinde aynı konvansiyonla `allocated_hours` türetilir; `duration_days` (inclusive) $\times$ `hours_per_day`; açık `allocated_hours` her zaman kazanır. Süre-only edit'lerde `date_deadline` delta saat kadar uzar → write hook successor cascade'ini tetikler.

2.5 Delay Impact (`_recalculate_delay_impacts`)

En yeni baseline'ın `line_ids` 'ı ile güncel görevleri karşılaştırır:

- `variance = allocated_hours - snapshot.allocated_hours`
- `variance > 0` → `impact = min(max(0, variance - max(0, snapshot.slack)), max(0, total_delay))` , status `critical_impact` veya `within_slack`

- `variance < 0` → negatif impact, `duration_reduced`
- `total_delay = critical_path_duration - baseline.project_duration`
- `_get_delay_impact_chain()` — başlangıcı kayan downstream'lerden en geç bitireni izleyerek "Task A (+3h) → Task B start +2h → Project finish +3h" metni üretir.
- Sonuçlar `project.task.delay.impact` satırlarına ve görev alanlarına yazılır; baseline yoksa her şey sıfırlanır.

2.6 Baseline Akışı

`action_create_critical_path_baseline(baseline_label=None)`:

1. `check_planner_manager` → `_recalculate_critical_paths()`
2. `revision_number = prev + 1`, `name = "v1.N" [- label]` (label `project.baseline.title` 'dan seçilir, UI'da zorunlu)
3. Header'a süre, imza, JSON snapshot + `_get_baseline_resource_vals()` (hook: çekirdekte 0, resource addon'da gerçek saat/maliyet)
4. `_create_snapshot_lines()` — her görev için donmuş satır (süre, tarihler, ES/EF/LS/LF, slack, `is_critical`, kaynak saat/maliyet)
5. `_recalculate_delay_impacts()` → `_recalculate_critical_path_changes()` → `_recalculate_history_comparisons()` (ardışık baseline çiftleri arasında süre/maliyet varyansı + CP giriş/çıkış listeleri ve özet metin)

2.7 Güvenlik — Planner Manager

- `res.groups.privilege` "Planner" altında `group_planner_manager` (internal user'a implied; admin+root üye).
- `check_planner_manager(env)`: `env.su` değilse ve grup üyeliği yoksa `AccessError`. **Tüm planner mutasyon yolları** bu kapıdan geçer:
- `project.task.write/create/unlink` — `PLANNER_GUARDED_FIELDS = {name, user_ids, allocated_hours, sequence, date_assign, date_deadline, depend_on_ids, dependent_ids, parent_id}` (create'te `name` hariç). Workflow alanları (`stage_id`, `state`, `description`, `tag_ids`) serbest — görev kapamak planner edit'i değildir. Compute/rollup alanları (`wbs_*`, `progress`, `date_done`) bilinçli olarak guard dışı.
- `project.task.dependency` tüm CRUD'u (reconcile sudo ile olduğu için okuma akışları etkilenmez).
- `project.write` — `planning_precision` ve `resource_calendar_id`.
- Baseline oluşturma, planner RPC metotları (`update_planner_task`, `update_planner_dependency`, `planner_wbs_*`, `planner_add_subtask`, kaynak addon'daki `planner_*` metotları).
- `planner_wbs_indent/outdent` iç `sudo()` yazışı kullanır — kapı metodun başında çalıştığı için tek koruma noktasıdır.
- `ir.model.access.csv`: tüm yeni modellere `base.group_user` tam CRUD; gerçek yetki uygulama seviyesinde kapıyla verilir.

2.8 WBS İşlemleri (Planner API)

- `_calculate_wbs_codes_map` — kökten recursive; kardeşler (`sequence`, `id`) ile sıralı; `wbs_sort_key` zero-pad.

- `_recalculate_wbs` — kod/seviye/sort_key/work_package/rollup'ları tek geçişte günceller, sadece değişenleri yazar. Tetikleyiciler:
create/write(parent_id|sequence|project_id|allocated_hours|progress)/unlink.
- `planner_add_subtask(name)` — "+" butonu: child yaratır, inline rename.
- `planner_wbs_indent()` — bir üstteki kardeşin son çocuğu olur; ilk kardeş indent edemez.
- `planner_wbs_outdent()` — eski parent'ın hemen altına yerleşir.
- `planner_wbs_move_before(before_id)` — aynı parent içi sıralama; **drag&drop asla reparent etmez** (başka daldaki hedef reddedilir).
- `_planner_wbs_normalize` — kardeş sequence'ları 10,20,30... olarak yazar.

2.9 Planner Server API'si (`project.project` / `project.task`)

Metot	Model	Dönüş
<code>action_open_planner_workspace()</code>	project	<code>ir.actions.client</code> → <code>project_critical_path.planner_workspace</code>
<code>get_planner_projects()</code>	project	Seçilebilir projeler + <code>search_view_id</code> + <code>has_resource_planning</code>
<code>get_planner_data(baseline_id, domain)</code>	project	WBS sıralı düz görev listesi + meta (users/roles/stages/parents) + proje (precision, hours_per_day, calendar) + <code>is_planner_manager</code> . <code>domain</code> filtreleri uygular ve eşleşenlerin WBS atalarını da korur . <code>baseline_id</code> ghost katmanını değiştirir.
<code>get_planner_baseline_history()</code>	project	Sadece saklı alanlardan okunur (ucuz)
<code>get_planner_baseline_summary(baseline_id)</code>	project	Baseline ↔ önceki satır karşılaştırması (delta saat/maliyet, CP giriş/çıkış, yeni/silinen görev)
<code>get_planner_detail()</code>	task	Quick Inspector verisi + seçenekler
<code>update_planner_task(values)</code>	task	Inspector/timeline yazışı — yukarıda §2.4
<code>update_planner_dependency(depends_on_id, relationship_type, lag, lag_unit)</code>	task	Kenar niteliği yazışı → <code>_schedule_dependents</code> + CPM recalc
<code>planner_add_subtask</code> / <code>planner_wbs_indent</code> / <code>outdent</code> / <code>move_before</code>	task	WBS işlemleri

Tarih serileştirme: `_serialize_planner_day` → YYYY-MM-DD, `_serialize_planner_dt` → YYYY-MM-DD HH:MM (user tz). `_local_day_to_utc` / `_local_dt_to_utc` ters yön (fallback saat 09:00/18:00; mevcut kayıta saat korunur).

Takvim payload'ı (`_planner_calendar_data`): projenin `resource_calendar_id` 'si — öğle arası ve tarih-sınırlı satırlar hariç attendance'lar + global/kaynak-sız leave'ler. Takvim yoksa `False` → frontend sürekli (compress'lenmemiş) zaman eksenini kullanır.

Odoo uyumluluk notları: - `create()` içinde `user_ids` + açık `date_assign` birlikte gelirse assignees sonraki write'a ertelenir — Odoo'nun "assigned today" damgalaması explicit planlanan başlangıcı ezmesin diye. - `update_planner_task` 'te `user_ids` önce yazılır; Odoo assignee boşaldığında `date_assign` 'ı sildiği için start korunup geri yazılır.

2.10 Frontend — `planner_workspace.js` (~3000 satır, OWL Component)

`project_critical_path.planner_workspace` client action'ı → `PlannerWorkspace` component.

Başlıca alt sistemler:

- **Sürekli timeline:** `rangeStart/End` buffer'lı eksen; kenara yaklaşıncı `extendRange` iki yönde büyür, sol genişlemede scroll telafisi. `computeRange`, `snapAnchor`, `origin`.
- **Folded timeline:** `_buildTimeMap` çalışma takviminden piksel↔ms haritası; `timeToX` / `xToTime` çalışma-dışı zamanı sıkıştırır (gri).
- **Ölçekler:** day/week/month; day ölçeğinde `planning_precision == "hour"` ise saat-hassasiyetli drag (`dt_start/dt_stop`), day precision gün-kuantumlu snap.
- **Bar drag/resize:** `onBarPointerDown` (move/left/right), 4px click eşiği, kenarda auto-scroll (`tickDragAutoScroll`, extraDx telafisi), `persistTaskDates` → `update_planner_task` → `reloadPlannerData` (seçim/collapse/inspector korunur).
- **Dependency okları:** `dependencyEdges` — görev çiftlerinden SVG path; FS/SS/FF/SF geometrisi ve `FS+2d` lag etiketi; ok tıklaması edge editör popover'ı açar (`openDepEditor/saveDepEdit` → `update_planner_dependency`).
- **Baseline katmanları:** ghost bar (`baselineStyle`), varyans kuyruğu (`varianceGeometry/Style`), Baseline History paneli (liste, detay, compare overlay, görev-atlama), Baseline Save (title seçimi).
- **Finish Variance Tail:** `finishTail` — done görevin planlanan bitişini geçen/altında kalan gerçek kapanışı gösteren kuyruk + `+Ng/-Ng` etiketi; gün-bazlı hesap.
- **Quick Inspector:** `loadInspector/saveInspector` — ad, tarihler, süre (gün ve saat modları), progress, stage, assignees (tag-picker), predecessors/successors, baseline karşılaştırması, delay impact.
- **WBS paneli:** satır listesi + collapse, indent/outdent butonları, "+" quick-create, inline rename, **yalnızca sıralama** yapan drag (`_wbsDragMove/_wbsDragEnd`), gantt ile senkron scroll.
- **Filtre/Group By:** `filterDomain` — status filtreleri (critical, delayed, completed, completed-late/early/on-time → stored `finish_variance_state`), metin/tarih alanları, Odoo search-view domain'i; server-side `get_planner_data(domain=...)`. Group by: project/stage/assignees/wbs_level (+resource'ken role/resource type).
- **Bar info:** `BAR_INFO_MAX` kadar seçilebilir alan (`barInfoText`), slack chip toggle, per-project persist.
- **Zoom:** per-scale density zoom, `fit`, `fitTimeline`, `goToday`, date-picker navigation.

2.11 Frontend — Native Gantt patch'i (`gantt_dependencies.js`)

`web_gantt` 'in `GanttModel`, `GanttRenderer`, `GanttRendererControls` ve `GanttConnector` 'ı patch'lenir:

- `GanttModel`: `dependency_field="depend_on_ids"` / `dependency_inverted_field="dependent_ids"` view'a `_get_view` ile enjekte edilir; `gantt_dependency_metadata` Json alanı kenar başına `{type, lag}` görsel metadata taşır (scheduling'i **etkilemez**).
- `gantt_dependency_geometry.js`: `dependencyGeometry(source, target, type, rtl)` — 4 tip için ok path'i; `visualDependency` metadata'yı görsel tipe çevirir.

- `DependencyDialog` + `DependencyMenu` : bağlantı tıklanınca tip/lag düzenleme → `set_gantt_dependency` / `remove_gantt_dependency` (row-level `SELECT ... FOR UPDATE` kilidi, access check, self-link ve cycle validasyonu; `write()` içinde orphan metadata temizliği ve `_has_cycle("depend_on_ids")` kontrolü).
- Kanban: `is_critical` kartlara `o_critical_path_kanban` class + uyarı rozeti (`critical_path_kanban.scss`).

2.12 View'lar (çekirdek)

- `project.project` form: header'a *Planner* (primary), *Calculate Critical Paths* ve *Create Baseline* (project manager grubu); Settings sayfasına *Planning* grubu (`planning_precision`, `resource_calendar_id`); notebook'a **Work Breakdown Structure, Critical Paths, Plan Baselines, Delay Impact** sayfaları.
- `project.task` form: WBS badge satırı + **Critical Path Schedule** sayfası (early/late/slack/critical + delay impact).
- Task list: `default_order="wbs_sort_key"`, WBS/rollup/critical kolonlar; search: WBS filtreleri + group-by'lar.
- `project_task_planner_search` : planner search bar için dedicated search view (critical/delayed/this-week/completed filtreleri).
- `project.baseline.title` list/form + `Project` ▶ `Configuration` ▶ `Baseline Titles` menüsü; `Planner` menüsü `project.menu_main_pm` altında.
- Modül ayrıştırma temizliği: eski `project_material_*` / `resource_*` XML kayıtları `<delete>` ile kaldırılır (upgrade'de orphan kalmaması için).

2.13 Testler & Migration (çekirdek)

- `tests/` : `test_critical_path` (CPM), `test_planning_precision`, `test_gantt_dependencies`, `test_finish_variance(_day)`, `test_planner_workspace`, `test_planner_filters`, `test_planner_security`, `test_planner_calendar`, `test_planner_certification`, `test_planner_wbs_actions`, `test_planner_tour`, `test_project_wbs`, `test_critical_path_kanban`,
- `gantt_geometry.test.mjs` (JS) + `planner_workspace_tour.js` (web tour).
- `migrations/19.0.1.44.0` : kaldırılan `project.material.requirement` modelinin SQL view/tablosunu drop eder.
- `migrations/19.0.9.0.0` : tarihli yaprak görevlere `allocated_hours` backfill + proje başına tek CPM recalc.

3. `project_resource_planning` — Kaynak Katmanı

3.1 Veri Modeli

Model	Amaç
<code>project.resource.role</code>	Kaynak rolü: <code>name</code> , <code>category</code> (<code>human equipment</code>), <code>priority</code> (1–5 Level, <code>hr.employee.priority</code> ile aynı Selection), <code>active</code>
<code>project.task.resource.requirement</code>	Görev başına rol ihtiyacı

Model	Amaç
<code>project.task.resource.assignment</code>	İhtiyaca bağlı gerçek kaynak rezervasyonu
<code>project.resource.rate.template</code>	Para birimi + saatlik planlama ücreti (opsiyonel <code>role_id</code>)
<code>project.resource.plan.summary</code>	Proje × rol toplamları (rebuild edilen özet)
<code>project.resource.planner</code> (+ <code>.line</code>)	Transient "Team Planner" — uygunluk listesi
<code>project.material.plan</code>	Taslak→Onaylı malzeme satırı → stock.picking

`project.task.resource.requirement`

- `task_id` (required, cascade), `project_id` (related, store), `role_id` (required), `level` (related → `role_id.priority`, store), `quantity` (default 1), `date_start/date_end`, `planned_hours`, `description`.
- Atama özeti (compute): `assigned_quantity` (distinct kaynak sayısı), `assigned_hours`, `assignment_status` → `waiting` (atama yok) / `partial` / `assigned` (miktar **ve** saat ihtiyacı karşılandı).
- Maliyet: `hourly_rate` (**snapshot** — template sonradan değişse eski planı bozmaz), `currency_id` (compute: proje `resource_currency_id` → ilk template currency → şirket), `planned_cost` = `qty` × `planned_hours` × `hourly_rate` (Monetary, store).
- `_resolve_hourly_rate()` : Önce role'e bağlı template, yoksa role'süz genel template; hiçbiri yoksa `None` → maliyet hesaplanmaz.
- `create()` : `date_start/end` görevin `date_assign/date_deadline` 'ından default'lanır; explicit `hourly_rate` yoksa resolve edilir. `role_id` onchange'i ve `task_id/role_id` değişen write'lar rate'i yeniden resolve eder (explicit rate ezilmez). Tüm create/write/unlink `project._recalculate_resource_plan()` tetikler.
- Aksiyonlar: `action_open_resource_assignments` (liste), `action_open_resource_planner` (transient planner form), `action_open_resource_timeline` (kategoriye göre employee/equipment gantt view + requirement penceresi domain'i).

`project.task.resource.assignment`

- `requirement_id` (required, cascade); `task_id` / `project_id` related store; `resource_category`, `requirement_role_id` related.
- `employee_id` **xor** `equipment_id`; `date_start/date_end` required.
- `planned_hours` (compute, store): kaynağın (yoksa şirketin) `resource_calendar_id.get_work_hours_count(start, end, compute_leaves=True)`; takvim yoksa düz saat farkı.
- `_check_assignment` constraint'leri:
- human ihtiyaç → tam 1 employee (equipment yasak) **ve** employee `resource_role_ids` içinde rolü taşımali
- equipment ihtiyaç → tam 1 equipment (employee yasak)
- `date_end` > `date_start`; atama penceresi requirement penceresi içinde
- aynı requirement içinde aynı kaynağa overlap yok
- eşzamanlı distinct kaynak sayısı ≤ `requirement.quantity`
- $\sum \text{assignment.planned_hours} \leq \text{requirement.planned_hours}$

project.resource.planner (TransientModel)

`create_for_requirement(requirement) → _build_lines()`:

- `human` → `resource_role_ids` $\ni$ `role` olan aktif employee'lar; `equipment` → tüm aktif ekipman.
- Requirement penceresiyle çakışan tüm assignment'lar (proje sınırı yok — kapasite kaynak bazlıdır).
- Satır başına `available_hours` (kaynak/şirket takvimi) ve `booked_hours` (pencereyle kesişim), `availability_status`: `fully_available` / `partially_available` / `unavailable` (`booked` $\geq$ `available`).
- `_order` = "priority desc, availability_order, resource_name".
- `action_assign_resource` → assignment form (default'larla),
`action_open_resource_timeline` → kaynağa filtreli gantt.

project.material.plan

- Zorunlu: `project_id`, `task_id` (aynı proje, domain+constrain), `product_id` (`is_storable`, `ondelete=restrict`), `planned_quantity` > 0 , `required_date` (default: görev `date_deadline` ı, yoksa today).
- `source/destination_location_id` (internal, compute default = proje `material_*_location_id`, satırda editable).
- Maliyet: `unit_cost` = `product_id.standard_price` (related), `planned_cost` = `qty` $\times$ `unit_cost`.
- İlişkiler: `move_ids` (`o2m stock.move.material_plan_line_id`),
`picking_ids` / `picking_id` / `picking_count` (compute), `parent_task_id` + `wbs_code` (related, store).
- **Lifecycle**: `draft` → `action_approve()` → `approved`. Onay: satırlar (`project`, `source`, `destination`) ile gruplanır; grup başına **tek standart stock.picking** (internal picking type — önce `source`'u kapsayan warehouse'un, yoksa şirketin internal'ı), satır başına **bir stock.move** (`material_plan_line_id` link, `origin` = "Material Plan / <proje>", `scheduled_date` = `min(required_date)`). `picking.action_confirm()` + `action_assign()` — rezervasyon, shortage, replenishment tamamen standart Odoo.
- Approved satırlar **immutable**: `write` → `UserError`, `unlink` → `UserError` (context `material_plan_force_unlink` admin/test kaçıışı).
- `action_open_transfers`: 1 picking → form, N → liste.

Diğer uzantılar

- `stock.move.material_plan_line_id` (m2o, set null).
- `stock.picking.material_plan_line_ids` (compute m2m) + `action_open_material_plan_lines` stat button.
- `hr.employee`: `resource_role_ids` (m2m role) + `priority` (0–5).
- `project.project`: `resource_rate_template_ids` (m2m, active domain),
`resource_currency_id` (template'ten senkron, readonly), `resource_cost_currency_id` (compute), `planned_resource_cost` = $\sum$ `requirement.planned_cost`,
`material_source/destination_location_id`, `material_picking_ids/count` (move linkinden resolve), `resource_requirement_ids`, `resource_assignment_ids`,

`resource_plan_summary_ids ; _recalculate_resource_plan()` requirement'ları role bazında toplayıp summary'yi yeniden kurar.

- `project.task : resource_requirement_ids , planned_resource_cost`.
- `project.critical.path.baseline._get_task_resource_snapshot` → görev requirement toplamı (saat/maliyet/currency) baseline satırına donar.
- `project.project._get_baseline_resource_vals` → baseline header'a `planned_resource_hours/cost + currency_id`.

3.2 Planner Server API'si (kaynak)

Metot	Açıklama
<code>project.planner_get_resource_board(window_start, window_end)</code>	Tüm uygun kaynaklar + penceredeki booking'ler; <code>occupancy = booked/available×100</code> (kaynak/şirket takvimi); personel priority desc, sonra ekipman
<code>task.get_planner_resources()</code>	Requirement'lar + atamalar + aktif roller + rate preview (currency, role→rate haritası) + görev tarihleri
<code>task.planner_save_requirement(values)</code>	Create/update (guard'lı); tarih default'ları görevden
<code>task.planner_delete_requirement(id)</code>	Sil (guard'lı, aidiyet kontrolü)
<code>task.planner_get_assignment_options(req_id, window_start, window_end)</code>	Transient planner'ı kullanıp aday listesi + aday başına pencere booking'leri (<code>mine , overlaps</code> flag'leri); default pencere = requirement ±2 gün
<code>task.planner_assign_resource(req_id, employee_id, equipment_id, date_start, date_end)</code>	Atama yaratır (guard'lı; tarihler requirement/görev default'larından, model constraint'leri geçerli)
<code>task.planner_estimate_assignment_hours(...)</code>	Aynı takvim hesabıyla önzileme saati
<code>task.planner_unassign_resource(id) / planner_update_assignment(id, start, end)</code>	Silme ve timeline move/resize

3.3 Frontend — `planner_workspace_resource.js` (~1265 satır)

`PlannerWorkspace.prototype` patch'i — ayrı dosya, çekirdeğe dokunmaz:

- **Resource Workspace modalı** (`openResourceWorkspace/loadResources`): görevin requirement listesi + durum rozetleri + requirement formu (`openRequirementForm/saveRequirement/deleteRequirement`) — rol, miktar, saat, tarihler, açıklama; `resFormRate/resFormCost` template'ten canlı maliyet önzilemesi.
- **Candidate timeline** (`resTl*`): requirement seçilince `planner_get_assignment_options`; aday başına booking bar'ları, availability dot'ları, yıldız/avatar, pencere navigasyonu (gün/hafta ölçek, today, date-pick).
- **Pending assignment** (`resTlPending*`): timeline'da sürükleyerek aralık seçimi → tahmini saat (`planner_estimate_assignment_hours`) → "Assign" commit'i

(`planner_assign_resource`); conflict/outside görsel durumu.

- **Mevcut atama aksiyonları:** bar seçimi → taşı/resize (`planner_update_assignment`) ve silme onayı (`planner_unassign_resource`).
- **Resource Board** (`board*`): çekirdek workspace'in ikinci modu — `planner_get_resource_board` ; personel/ekipman grupları (collapsible), occupancy, booking bar'ları + tooltip'ler, navigasyon; view-only.
- `openMaterialPlan` → projenin material plan aksiyonu.
- **Material Transfer Badge** (`fields/material_transfer_badge.js`): `picking_count` integer widget'ı — 0 → boş, 1 → picking form, >1 → liste; `action_open_transfers` 'a delege.

3.4 View'lar (kaynak)

- Proje form: header'a *Material Plan + Transfers* (count>0); notebook'a **Resource Plan** (rate template'lar, currency, planlanan maliyet, summary, requirement listesi, atama listesi) ve **Material Plan** (default lokasyonlar + transfer sayacı) sayfaları.
- Görev form: **Resource Requirements** sayfası — editable requirement listesi, rol/level(priority widget)/miktar/tarih/saat/rate/maliyet/ atama durumu + *Plan Resources / Assign Resources* butonları.
- Assignment list/form + employee/equipment **gantt view'ları** (`default_group_by` employee/equipment, `color="task_id"`).
- `hr.employee` form: *Resource Planning* grubu (roller, priority).
- `project.resource.planner` form: transient Team Planner (satır başına *Assign / Timeline*).
- `project.resource.role` + `project.resource.rate.template` list/form + **Project** ► **Configuration** menüleri.
- `project.material.plan` list (`editable="bottom"` , draft/approved dekorasyonları), form (header *Approve* + statusbar), search (ürün/görev/proje/lokasyon/state filtreleri + group-by'lar), liste Action menüsünde **Approve** server action'ı.
- `stock.picking` form: *Material Plan* stat button + ilişkili satırlar.

3.5 Testler (kaynak)

- `test_resource_planning.py` (~456 satır): requirement/assignment akışı, constraint'ler, planner board, saat hesapları.
- `test_material_plan.py` (~389 satır): onay → picking/move oluşumu, immutability, lokasyon default'ları.

4. Cross-Cutting Konular

4.1 Context Flag'leri

Flag	Etki
<code>cp_skip_recalc</code>	<code>project.task</code> write'ında CPM recalc'ı atlar
<code>cp_skip_auto_schedule</code>	<code>_schedule_dependents</code> cascade'ini atlar (kendi shift write'ları bunu taşır)
<code>material_plan_force_unlink</code>	Approved material plan satırını silebilme (UI'da yok, admin/test)

4.2 Tasarım Kararları (koddan çıkarılan)

- **depend_on_ids tek gerçek kaynak:** Odoo native alanı; `project.task.dependency` sadece kenar niteliği ve lazy reconcile edilir.
- **allocated_hours = sürenin tek kaynağı:** CPM, delay impact ve baseline hep bunu okur; pencere→saat senkronu seed-once kuralıyla yapılır.
- **Değişmeyen satırlar UPDATE edilmez:** CPM ve delay-impact yazışları toleranslı diff yapar — her drag'de tam tablo rewrite önlenir.
- **Baseline gerçekten immutable:** model seviyesinde write/unlink bloklü; history karşılaştırmaları donmuş snapshot'lardan türetilir, hesaplanmaz.
- **WBS ≠ schedule:** parent'lar grafikte düğüm değildir; pencereleri çocuklardan roll-up'tır ve cascade'den muaf'tır.
- **Stok yeniden icat edilmez:** Material Plan onayı standart picking/move üretir; sonrası Odoo.
- **Tek yönlü bağımlılık:** çekirdek asla kaynak addon'unu bilmez — sadece boş hook'lar ve `has_resource_planning` bayrağı.
- **sudo() kullanımı:** `_ensure_dependency_records` (reconcile), CPM path create/delete; `planner_wbs_*` iç yazışları (kapı metod başında).

4.3 Gözlemlenen Potansiyel Sorunlar

1. **_local_day_to_utc import eksikliği** — `project_resource_planning/models/project_planner.py` içinde `planner_save_requirement` `_local_day_to_utc` çağırır ama import bloğu yalnızca `_local_dt_to_utc` 'yi içerir. JS `saveRequirement` `date_start / date_end` her zaman gönderir; **kullanıcı requirement formunda özel tarih girerse NameError oluşur**. Boş bırakılırsa (`false`) erken dönüş nedeniyle crash olmaz — gizli bug.
2. **_compute_wbs_code_and_level performansı:** `depends child_ids` geniş — her compute'da projenin tüm görevleri search edilir; büyük projelerde write yükü artabilir (rollup ile birlikte `_recalculate_wbs` de tekrar eder — çift hesap olasılığı).
3. **action_open_planner_workspace / planner API erişimi:** okuma yolları (`get_planner_data` vb.) group kontrolü yapmaz — tüm internal kullanıcılar planner verisini okuyabilir (tasarım: read-only planner); yazışlar kapalıdır.
4. **get_planner_data ölçeklenebilirliği:** tüm görevler tek payload'da serileştirilir; binlerce görevli projelerde boyut/yazışma maliyeti.
5. **Cycle kontrolü iki yerde:** `_get_task_dependency_graph` (UserError) ve `gantt write()` (`_has_cycle` → ValidationError); planner `depend_on_ids` yazışında cycle durumunda hangi hatanın yüzeye çıktığı yola bağlı.
6. **MaterialTransferBadge template adı** `project_critical_path.*` namespace'inde ama asset `project_resource_planning` 'de — fonksiyonel sorun yok, isimlendirme tutarsızlığı.

5. Dosya Haritası

```
project_critical_path/
├─ models/
│   ├─ project_project.py          CPM, delay impact, dependency graph, auto-schedule
│   └─ project_planner.py         Planner RPC'leri (project + task), tz dönüşümleri, gü
```

```

├─ project_wbs.py          WBS kod/rollup + planner WBS aksiyonları
├─ project_task.py        Görev alanları, guarded write, date_done, parent wino
├─ project_task_dependency.py Kenar nitelikleri (FS/SS/FF/SF + lag)
├─ project_task_gantt.py   Native gantt dependency metadata + RPC
├─ project_critical_path.py Sonuç modeli
├─ project_critical_path_baseline.py Baseline + baseline.line (immutable)
├─ project_critical_path_change.py CP giriş/çıkış kaydı
├─ project_task_delay_impact.py Etki analizi satırı
├─ project_baseline_title.py Baseline başlık sözlüğü
├─ static/src/
├─   └─ planner/planner_workspace.{js,xml,scss}  OWL Planner Workspace
├─   └─ gantt_dependencies.{js,xml,scss}         Native gantt patch + dialog
├─   └─ gantt_dependency_geometry.js             Ok geometrisi (saf fonksiyon)
├─   └─ scss/critical_path_kanban.scss
├─ views/ project form/task/kanban/planner/baseline-title XML'leri
├─ security/ group_planner_manager + erişim csv
├─ tests/ 14 python test + 1 mjs + tour
├─ migrations/ 2 post-migrate script
├─ demo/project_wbs_demo.xml

project_resource_planning/
├─ models/
├─   └─ project_resource.py          Role + Requirement + PlanSummary
├─   └─ project_resource_assignment.py Atama + takvim saati + constraint'ler
├─   └─ project_resource_rate.py     Rate template
├─   └─ project_resource_planner.py  Transient Team Planner (+line)
├─   └─ project_material_plan.py     Draft→Approved → stock.picking
├─   └─ project_planner.py           Resource Board + planner resource RPC'leri
├─   └─ project_project.py           Proje alanları + summary rebuild + baseline hook
├─   └─ project_task.py              Görev alanları + planner payload hook
├─   └─ project_critical_path_baseline.py Task-level maliyet snapshot hook
├─   └─ hr_employee.py               resource_role_ids + priority
├─   └─ stock_move.py / stock_picking.py İzlenebilirlik linkleri
├─ static/src/
├─   └─ planner/planner_workspace_resource.{js,scss} Resource Workspace + Board patch
├─   └─ fields/material_transfer_badge.{js,xml,scss}
├─ views/ resource/role/rate/material/assignment/employee/gantt XML'leri
├─ security/ erişim csv

```

Hazırlanma tarihi: 2026-10-03 · Kapsam: çekirdek v19.0.9.0.0 + kaynak v19.0.1.0.0 · Kaynak: doğrudan kod analizi

